

Agent Mesh SRE

Reference Architecture & Blueprint

Monitor · Reason · Act · Learn

Version 1.0 · May 2026 · Confidential

1. Executive Summary

Agent Mesh SRE is a reference implementation of an agentic AI system for autonomous Kafka cluster operations. Four specialised agents collaborate through a shared event bus, each executing the Monitor → Reason → Act → Learn (MRAL) loop independently while surfacing human-in-the-loop approval gates for high-impact actions. This document extracts the core architectural patterns into a reusable blueprint that teams can adapt for other domain-specific agentic workflows.

Key capabilities demonstrated:

- Decoupled, event-driven agent coordination via Kafka topics (MOCK or REAL mode)
- Typed MCP Tool Registry with JSON Schema contracts, policy tags, and per-agent binding
- AsyncAPI 3.0 programmatic topic specifications for documentation and validation
- Real-time dashboard with ReactFlow topology, SSE state streaming, and particle animations
- Human-in-the-loop approval gate: async Promise resolution with audit trail
- Adaptive learning loop: lesson records that adjust detection thresholds over time
- Email summaries covering the full MRAL execution trace per scenario

2. System Context

The system sits between a live (or mocked) Kafka cluster and human operators. It continuously monitors broker metrics, detects anomalies, reasons about root causes, applies remediation actions (with or without human approval), and persists lessons to influence future decisions.

2.1 Actors and External Interfaces

Actor / System	Interface	Direction	Notes
Kafka Broker (MOCK/REAL)	KafkaJS consumer/producer	Bidirectional	Metrics in, actions out
Human Operator	Browser dashboard + Approval Gate modal	Bidirectional	Approve / reject tool calls
Slack	sre.notifySlack MCP tool	Outbound	Incident notifications
ITSM (PagerDuty / ServiceNow)	sre.openITSMTicket MCP tool	Outbound	Ticket creation
Email (SMTP)	sre.sendEmailSummary / Nodemailer	Outbound	Full MRAL trace per scenario
Next.js frontend	SSE /api/mesh/stream + REST endpoints	Bidirectional	Real-time state + control

3. The MRAL Loop Pattern

Every agent in the mesh executes the same four-phase cognitive loop. This pattern is the primary reusable primitive — it can be extracted and embedded into any agentic system that needs structured, auditable decision cycles.

3.1 Phase Definitions

Phase	State Value	Description	Triggers next phase when...
-------	-------------	-------------	-----------------------------

MONITOR	monitor	Consumes incoming events/metrics from the bus. No decisions made.	Anomaly threshold exceeded
REASON	reason	LLM call (or deterministic logic) produces a structured ReasoningOutput.	Reasoning confidence $\geq$ threshold
ACT	act	Executes the recommended MCP tool call, optionally gated by human approval.	Tool call completes (success or failure)
LEARN	learn	Persists a LessonRecord; adjusts future thresholds based on outcome.	Lesson written to bus
AWAITING	awaiting	Paused, pending human approve/reject decision.	Human decision received
REPLAYING	replaying	Re-executing a prior approved action from audit history.	Replay complete
IDLE	idle	No active scenario; agent is healthy but dormant.	New event arrives

3.2 Core TypeScript Interfaces

MralPhase union type:

```
type MralPhase =  
  | "idle" | "monitor" | "reason"  
  | "awaiting" | "act" | "learn" | "replaying";
```

ReasoningOutput — structured output from the REASON phase:

```
interface ReasoningOutput {  
  rootCause:      string;  
  confidence:     number;           // 0-1  
  kafkaFeatureCited: string;       // e.g. "KIP-848"  
  rebalanceState: string;  
  controllerEpoch: number;  
  crossCorrelation: CrossCorrelationSignals;  
  recommendedAction: string;       // maps to an MCP tool id  
  requiresApproval: boolean;  
  rationale:      string;  
  lessonsCited:   string[];        // prior lesson ids that influenced this  
}
```

ActionResult — outcome from the ACT phase:

```
interface ActionResult {  
  approved:      boolean;  
  outcome:       "success" | "failure" | "skipped";  
  detail:        string;  
  lagBefore:     number;  
  lagAfter:      number;  
  toolCalled:    string;  
  clusterMutation: string;  
}
```

3.3 Agent State Interface

```
interface AgentState {
  id: AgentId; // "intake" | "monitor" | "writer" |
"notification"
  name: string;
  status: AgentStatus; // "online" | "reasoning" | "acting" | "awaiting-
approval"
// | "crashed" | "replaying" | "idle"
  mralPhase: MralPhase;
  color: string; // hex, used in dashboard node border
  lastAction: ActionResult | null;
  reasoning: ReasoningOutput | null;
  restartCount: number;
}
```

4. Agent Roster

Four agents are pre-wired in this reference implementation. Each is bound to a specific set of MCP tools and consumes/produces specific Kafka topics.

Agent ID	Responsibility	Consumes	Produces	Bound Tools
intake	Route raw ops requests to the metrics pipeline	ops.requests.v1	ops.kafka.metrics.v1	kafka.scaleConsumers, kafka.checkpointShareGroup
monitor	Detect anomalies; trigger REASON phase	ops.kafka.metrics.v1	ops.incidents.v1, ops.lessons.v1	kafka.ackControllerFailover, kafka.suppressRebalancePage, sre.draftIncidentReport
writer	Generate and publish incident reports	ops.incidents.v1	ops.actions.audit.v1	sre.draftIncidentReport, sre.sendEmailSummary
notification	Fan out alerts to Slack / ITSM / email	ops.actions.audit.v1	ops.notifications.v1	sre.notifySlack, sre.openITSMTicket, sre.sendEmailSummary

5. MCP Tool Registry

The MCP Tool Registry is the second primary reusable primitive. It provides typed, policy-annotated, agent-scoped tool definitions. Each tool carries a JSON Schema contract for both input and output, enabling runtime validation, documentation generation, and policy enforcement without extra infrastructure.

5.1 Tool Definition Schema

```
interface MCPToolDef {
  id: string; // e.g. "kafka.scaleConsumers"
  description: string;
  inputSchema: JsonSchema; // validated before execution
  outputSchema: JsonSchema; // validated after execution
  policyTags: string[]; // e.g. ["cluster-mutation", "safe-to-replay"]
  requiresApproval: boolean; // true → triggers Approval Gate
  execute: (params: unknown, ctx: ToolContext) => Promise<unknown>;
}
```

```

}

interface ToolContext {
  agentId: AgentId;
  scenarioId: string;
  ts: number;
  broker: BrokerState;
}

```

5.2 Tool Inventory

Tool ID	Category	Approval?	Policy Tags	Key Side Effect
kafka.scaleConsumers	Kafka	Yes	cluster-mutation, safe-to-replay	oc scale deploy/... --replicas=N
kafka.ackControllerFailover	Kafka	Yes	cluster-mutation	Triggers KRaft re-election
kafka.checkpointShareGroup	Kafka	No	read-only, safe-to-replay	Saves share-group offset checkpoint
kafka.suppressRebalancePage	Kafka	No	read-only	Suppresses false-positive alert page
sre.draftIncidentReport	SRE	No	write, idempotent	Writes structured incident record
sre.notifySlack	SRE	No	notify, idempotent	Posts message to Slack webhook
sre.openITSMTicket	SRE	No	notify, idempotent	Creates PagerDuty / ServiceNow ticket
sre.sendEmailSummary	SRE	No	notify, idempotent	Sends MRAL trace email via SMTP

5.3 Per-Agent Tool Binding

Each agent is constrained to a subset of the tool registry via `AGENT_TOOLS`. This enforces the principle of least privilege — an agent can only call the tools it explicitly owns.

```

const AGENT_TOOLS: Record<AgentId, string[]> = {
  intake: ["kafka.scaleConsumers", "kafka.checkpointShareGroup"],
  monitor: ["kafka.ackControllerFailover", "kafka.suppressRebalancePage",
    "sre.draftIncidentReport"],
  writer: ["sre.draftIncidentReport", "sre.sendEmailSummary"],
  notification: ["sre.notifySlack", "sre.openITSMTicket", "sre.sendEmailSummary"],
};

```

6. Human-in-the-Loop Approval Gate

Any tool with `requiresApproval: true` is intercepted before execution. The agent transitions to the `AWAITING` phase, pausing the MRAL loop, and an `ApprovalRequest` is broadcast on the SSE stream. The frontend renders a modal; the human's approve/reject decision resolves the pending Promise, resuming the loop.

6.1 ApprovalRequest Interface

```

interface ApprovalRequest {
  id: string; // uuid
  scenarioId: string;
}

```

```

agent:      AgentId;
toolCall: {
  tool:      string;          // MCP tool id
  params: Record<string, unknown>;
};
ts:         number;
}

```

6.2 Approval Flow

- 1. Agent calls executeTool(tool, params) — tool has requiresApproval: true.
- 2. Mesh creates an ApprovalRequest and registers a pending Promise resolver.
- 3. ApprovalRequest is pushed onto pendingApprovals and broadcast via SSE.
- 4. Dashboard renders the ApprovalGate modal; agent transitions to awaiting.
- 5. Operator clicks Approve or Reject → POST /api/mesh/approve {id, decision}.
- 6. Server resolves the Promise with the decision; agent resumes from ACT phase.
- 7. If rejected, agent transitions directly to LEARN with outcome: 'skipped'.
- 8. AuditRecord written: type='approval', includes decision and who approved.

7. Event Bus & AsyncAPI Topic Topology

All inter-agent communication flows through Kafka topics. In MOCK mode, an in-process event emitter substitutes for a real broker, enabling local demos without cluster infrastructure. In REAL mode, KafkaJS connects to an actual broker.

7.1 Topic Specifications

Topic	Partitions	Retention	Producer Agent	Consumer Agent(s)	Purpose
ops.requests.v1	3	7 days	External / API	intake	Incoming SRE ops requests
ops.kafka.metrics.v1	6	24 hours	intake	monitor	Broker metric snapshots
ops.incidents.v1	3	30 days	monitor	writer	Detected anomalies + reasoning
ops.actions.audit.v1	3	90 days	writer	notification	Completed action audit records
ops.lessons.v1	1	365 days	monitor/writer	monitor	Learned lessons for threshold tuning
ops.notifications.v1	3	7 days	notification	Dashboard	Fan-out notification records
ops.approvals.v1	1	24 hours	mesh broker	All agents	Approval decisions (REAL mode)

7.2 MOCK/REAL Mode Bridge

The KafkaBridge abstraction wraps both modes behind a common interface. Set KAFKA_MODE=REAL in environment to connect to a live cluster; the default MOCK mode uses the in-process event bus.

```
interface KafkaBridge {
  publish(topic: string, value: unknown): Promise<void>;
  subscribe(topic: string, handler: (msg: unknown) => void): void;
  mode: "MOCK" | "REAL";
}
```

7.3 Event Bus Singleton (globalThis pattern)

To survive Next.js hot-reload without losing subscriptions, the event bus is stored on globalThis. This is a critical pattern for any SSE-based system running under Next.js dev mode or serverless environments where module caches reset.

```
// lib/event-bus.ts
const g = globalThis as typeof globalThis & { __meshBus?: EventBus };
if (!g.__meshBus) g.__meshBus = new EventBus();
export const bus = g.__meshBus;
```

8. SSE Streaming Architecture

The dashboard receives all state updates via a single Server-Sent Events (SSE) connection to `/api/mesh/stream`. The server sends a full state snapshot on connect, then incremental events (audit records, particles, toasts, approvals) as they occur. Keep-alive pings prevent proxy timeouts.

8.1 BusEvent Discriminated Union

```
type BusEvent =
  | { type: "state"; /* full MeshClientState snapshot */ }
  | { type: "audit"; record: AuditRecord }
  | { type: "toast"; message: string; kind:
    "info"|"success"|"warning"|"error" }
  | { type: "particle"; edgeId: string; fromNode: string; toNode: string }
  | { type: "notification"; record: NotificationRecord }
  | { type: "lesson"; record: LessonRecord }
  | { type: "approval"; request: ApprovalRequest }
  | { type: "ping" };
```

8.2 Client Hook (useMeshStream)

The React hook `useMeshStream()` encapsulates `EventSource` lifecycle management, exponential-backoff reconnection, and a `useReducer` for all state transitions. It returns the full `MeshClientState` plus imperative API helpers.

Export	Type	Description
<code>state</code>	<code>MeshClientState</code>	Full reactive state tree
<code>trigger(id)</code>	<code>async (scenarioId: string) => void</code>	POST <code>/api/mesh/scenario</code>
<code>approve(id, decision)</code>	<code>async (...) => void</code>	POST <code>/api/mesh/approve</code>
<code>agentAction(id, action)</code>	<code>async (...) => void</code>	POST <code>/api/mesh/agent (kill/restart)</code>
<code>reset()</code>	<code>async () => void</code>	POST <code>/api/mesh/reset</code>

9. Scenario Framework

Each runnable scenario is a data-driven record. Adding a new scenario requires only a new entry in the SCENARIOS array — no new code paths. The scenario drives the MRAL loop, determines which Kafka feature to cite, and controls the approval gate.

9.1 Scenario Interface

```
interface Scenario {
  id:          string;
  label:       string;
  description: string;
  kafkaFeature: string;    // cited in ReasoningOutput
  requiresApproval: boolean;
  icon:        string;
}
```

9.2 Built-in Scenarios

ID	Label	Kafka Feature	Approval?	Description
lag-spike	Consumer Lag Spike	KIP-848	Yes	High lag on consumer group; scales replicas
controller-failover	KRaft Controller Failover	KRaft	Yes	Controller epoch bump; triggers re-election
share-group	Share Group Rebalance	KIP-932	No	Checkpoints share-group offsets during rebalance
benign-rebalance	False-Positive Suppression	KIP-848	No	Detects and suppresses benign rebalance pages

10. Audit Log & Lessons Learned

10.1 AuditRecord Interface

```
interface AuditRecord {
  id:      string;
  ts:      number;
  agent:   AgentId;
  type:    "publish"|"consume"|"reasoning"|"tool-call"|"approval"
          | "lesson"|"notification"|"agent-kill"|"agent-restart"
          | "replay-start"|"replay-complete";
  summary: string;
  detail?: unknown;    // full payload for debugging
}
```

10.2 LessonRecord Interface

```
interface LessonRecord {
  id:          string;
  ts:          number;
  scenarioId:  string;
  actionTaken: string;
```



```

effective:      boolean;
lagBefore:      number;
lagAfter:       number;
adjustedThreshold: number; // new detection threshold for this scenario class
notes:          string;
}

```

Lessons are replayed on agent start-up (or scenario trigger) to prime detection thresholds. The monitor agent's REASON phase queries the lesson store for the most recent relevant lesson and includes its id in lessonsCited.

11. REST API Surface

Endpoint	Method	Description	Auth Required
/api/mesh/stream	GET	SSE stream of all mesh events	No (same-origin)
/api/mesh/scenario	POST	Trigger a scenario by id	No (same-origin)
/api/mesh/approve	POST	Submit approve/reject for pending request	No (same-origin)
/api/mesh/agent	POST	Kill or restart a named agent	No (same-origin)
/api/mesh/reset	POST	Reset all mesh state to initial	No (same-origin)
/api/mesh/snapshot	GET	Full state snapshot (JSON)	No (same-origin)
/api/email/test	POST	Send a test MRAL summary email	No (same-origin)

12. Gap Analysis

The following gaps exist between the reference implementation and a production-grade deployment. Each is categorised by priority.

Gap	Priority	Impact	Recommended Fix
No PolicyEngine — requiresApproval is hard-coded per tool	High	Cannot implement dynamic RBAC or tenant-level approval policies	Extract policy evaluation to a pluggable PolicyEngine interface
LLM calls are simulated — no real LLMAdapter	High	REASON phase uses deterministic mocks; no actual AI reasoning	Implement LLMAdapter interface; inject Anthropic/OpenAI SDK
Audit log is in-memory — lost on server restart	High	No persistence; audit trail unavailable after restart	Add Prisma/Postgres AuditRecord table; flush on write
No authentication on API routes	High	Any client can trigger scenarios or approve actions	Add NextAuth session check to all /api/mesh/* routes
KafkaJS REAL mode not wired to scenario runner	Medium	REAL mode consumers exist but scenario runner uses bus directly	Route scenario events through KafkaJS producer in REAL mode
No health/readiness endpoint	Medium	Cannot integrate with Kubernetes probes or load balancers	Add /api/health returning agent statuses and broker connectivity
Metrics collector is simulated	Medium	Metrics are generated synthetically; no real broker polling	Implement KafkaAdminClient-based MetricsCollector in REAL mode
Single-tenant — no workspace isolation	Low	One mesh instance per deployment; cannot share infrastructure	Add tenantId to all state structures; namespace Kafka topics
No rate limiting or replay deduplication	Low	Multiple rapid triggers can produce duplicate scenario runs	Add Redis-backed idempotency key check on scenario trigger
Email SMTP optional but not gracefully degraded in logs	Low	smtp_not_configured silently returns; operators may not notice	Add structured warning log entry when SMTP is unconfigured

13. Extension Guide

13.1 Adding a New Agent

- Add a new AgentId literal to the union type in types.ts: | "my-agent"
- Add initial AgentState to the agents array in mesh.ts
- Define allowed tools in AGENT_TOOLS["my-agent"] in mcp.ts
- Add a NODE_POS entry in AgentCanvas.tsx for layout positioning
- Wire the agent to consume from an existing topic or add a new one in asyncapi.ts
- Implement the agent loop calling runMraiLoop(agentId, scenarioId, ...) in mesh.ts

13.2 Adding a New MCP Tool

- Add a new entry to MCP_TOOLS in mcp.ts with id, description, inputSchema, outputSchema, policyTags, requiresApproval, and execute
- If the tool requires cluster access, inject the broker reference via ToolContext
- Add the tool id to AGENT_TOOLS for the relevant agent(s)
- If requiresApproval: true, the Approval Gate will automatically intercept it — no extra wiring needed
- Write an AuditRecord in the execute function for observability

13.3 Adding a New Scenario

- Add a new Scenario object to SCENARIOS in types.ts (or a scenarios.ts file)
- Add a matching case to the scenario runner switch statement in mesh.ts
- Define the synthetic metric payload the scenario will inject
- The scenario button will appear automatically in the Dashboard sidebar

13.4 Replacing the Mock LLM with a Real One

The REASON phase currently uses deterministic logic keyed on scenario id. To use a real LLM:

```
// Implement this interface in lib/llm.ts
interface LLMAdapter {
  reason(context: MetricSnapshot, lessons: LessonRecord[]):
  Promise<ReasoningOutput>;
}

// Swap the import in mesh.ts:
// import { reason } from "../llm"; // was: import { deterministicReason } from
// "../scenarios";
// Pass ANTHROPIC_API_KEY or OPENAI_API_KEY via .env.local
```

14. Deployment Tiers

Tier	Kafka Mode	LLM	Storage	Auth	Use Case
Demo / Local	MOCK	Simulated	In-memory	None	Quick demo, conference talks,

					portfolio
Development	MOCK	Real API	Postgres (dev)	JWT	Feature development with real reasoning
Staging	REAL	Real API	Postgres	NextAuth	Full integration testing on live cluster
Production	REAL	Real API	Postgres + S3	NextAuth + RBAC	24x7 autonomous Kafka SRE

14.1 Environment Variables Reference

Variable	Required	Default	Description
KAFKA_MODE	No	MOCK	MOCK REAL — switches broker bridge
KAFKA_BROKERS	REAL	localhost:9092	Comma-separated broker addresses
ANTHROPIC_API_KEY	No	—	Required if LLMAadapter uses Claude
OPENAI_API_KEY	No	—	Required if LLMAadapter uses GPT
SMTP_HOST	No	—	Email summary SMTP server
SMTP_PORT	No	587	SMTP port
SMTP_USER	No	—	SMTP credentials
SMTP_PASS	No	—	SMTP password
SMTP_FROM	No	—	Sender address for email summaries
NEXTAUTH_SECRET	Prod	—	JWT signing secret
DATABASE_URL	Prod	—	Postgres connection string (Prisma)

15. Production Hardening Checklist

Security

- Add NextAuth authentication to all /api/mesh/* routes
- Implement RBAC: restrict scenario triggers and approvals by role
- Rotate NEXTAUTH_SECRET and SMTP_PASS via secrets manager
- Enable HTTPS-only (force-ssl) in Next.js config
- Sanitise all SSE payloads before broadcasting to prevent XSS

Reliability

- Persist AuditRecords to Postgres — in-memory log is lost on restart
- Add circuit breaker around LLM calls (timeout + retry with backoff)
- Implement agent watchdog: auto-restart crashed agents after N seconds
- Add /api/health endpoint returning agent statuses and broker mode
- Set up Kafka topic auto-creation with correct partition/retention config

Observability

- Export MRAL phase transition metrics to Prometheus/Datadog
- Add structured logging (JSON) with correlationId per scenario run
- Alert on incidentQueueDepth > threshold
- Dashboard: add per-agent latency percentile panels (P50/P99)
- Trace approval gate wait time as a metric

Scalability

- Replace globalThis event bus with Redis Streams for multi-replica deployments
- Add consumer group for notification agent in REAL Kafka mode
- Implement topic namespace per tenantId for multi-tenant support
- Rate-limit scenario trigger endpoint (Redis-backed idempotency key)
- Move heavy LLM calls off the API route thread to a worker queue

16. Repository File Structure

```

agent-mesh-sre/
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── mesh/
│   │   │   │   ├── stream/route.ts    ← SSE stream endpoint
│   │   │   │   ├── scenario/route.ts  ← trigger scenario
│   │   │   │   ├── approve/route.ts   ← human approval
│   │   │   │   ├── agent/route.ts     ← kill / restart agent
│   │   │   │   ├── reset/route.ts     ← reset mesh state
│   │   │   │   ├── snapshot/route.ts  ← full state snapshot
│   │   │   │   └── email/test/route.ts ← test email endpoint
│   │   ├── layout.tsx
│   │   └── page.tsx
│   ├── components/
│   │   ├── AgentCanvas.tsx             ← ReactFlow mesh topology
│   │   ├── Dashboard.tsx              ← full dashboard layout
│   │   └── useMeshStream.ts            ← SSE client hook
│   └── lib/
│       ├── types.ts                   ← ALL domain interfaces
│       ├── mesh.ts                    ← MRAL loop engine
│       ├── mcp.ts                     ← MCP Tool Registry
│       ├── event-bus.ts               ← globalThis SSE bus
│       ├── asyncapi.ts                ← AsyncAPI 3.0 specs
│       └── emailer.ts                 ← Nodemailer + HTML template
├── next.config.mjs
├── tailwind.config.ts
└── package.json
  
```